Systèmes Embarqués Connectés

Module SLUE604

Cours de deuxième semestre

L3 Electronique / M1 Optiq

Pré-requis : SPUE402 2ème année
SLEE600 3ème année

François Verdier

LEAT UMR CNRS 7248







Systèmes Embarqués Connectés
Plan du cours

1. Introduction

2. L'outil de développement IDE ARDUINO

1. **Premier programme**
2. **Deuxième programme**
3. **Troisième programme…**

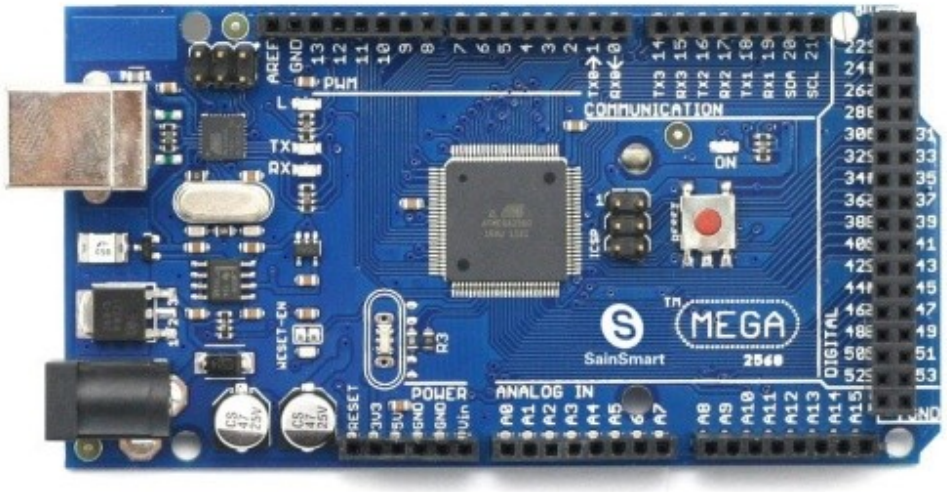
3. Les périphériques particuliers

1. **Les liaisons série**
2. **Les timers**
3. **Les sortie PWM**
4. **Les sorties de type « Servo »**

4. Les cartes spécialisés

1. La carte « RGD LED Matrix Driver »
2. La carte « Télémètre HC-SR04 »
3. Le shield « SD Card v4.3 »
4. La carte « UART Camera »
5. La carte « ArduCAM »
6. La carte « UART WIFI v2 »
7. Le protocole HTTP

1. Introduction



Carte Arduino à micro-contrôleur ATmega2560

Les objectifs de cette Unité d'Enseignement

- Nous allons faire une série de cours et de projets dont les objectifs seront d'utiliser l'IDE Arduino pour programmer le micro-contrôleur ATmega2560 en langage sketch (un langage très proche du langage C ou C++)
- Nous allons également voir quelques périphériques relativement classiques comme les liaisons série, les timers, les sorties PWM et quelques autres vraiment spécifiques (la matrice LED RGB 8x8, la caméra, le shield SD card, l'interface UART Wifi…) et programmer des applications qui utilisent ces dispositifs

Organisation des cours

- Vous allez avoir en principe une demi-douzaine d'heures de cours et environ 25 heures de projets qui seront dédiées à cet enseignement
- Le projet que vous allez faire sera noté, en fonction des avancées que vous aurez pu faire en cours de séances
- A l'issue de ce projet, vous devrez faire une présentation orale de ce que vous aurez fait. Cette présentation sera notée
- En plus de cela, nous noterons l'attitude et la participation semaine après semaine. L'ensemble de ces notations constituera une note globale.
- Cette note globale sera la seule note de cet enseignement

Le projet que vous devrez faire

- En projet, vous utiliserez des cartes Arduino Mega, avec des cartes d’interfaçage dédiées de type Grove mega shield



+



Le projet que vous devrez faire

- Vous apprendrez à programmer les micro processeurs AVR **correctement** en langage **sketch (c'est-à-dire quasiment en C++)**, et donc en utilisant l'environnement de développement Arduino IDE
- Vous serez confrontés à différentes difficultés, dont la première sera d'essayer de comprendre le fonctionnement des cartes spécifiques que vous devrez utiliser (la carte caméra UART, l'interface UART – WiFi…).

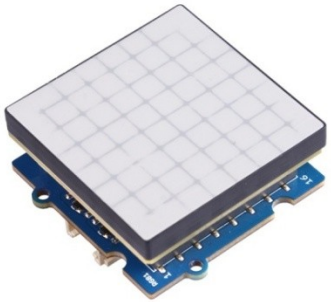
Le projet que vous devrez faire



+



+



+



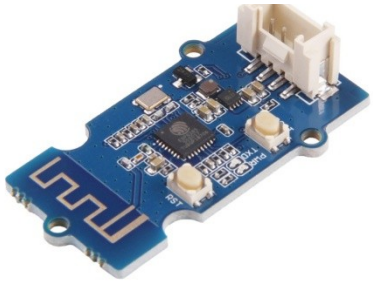
+



+



+



+



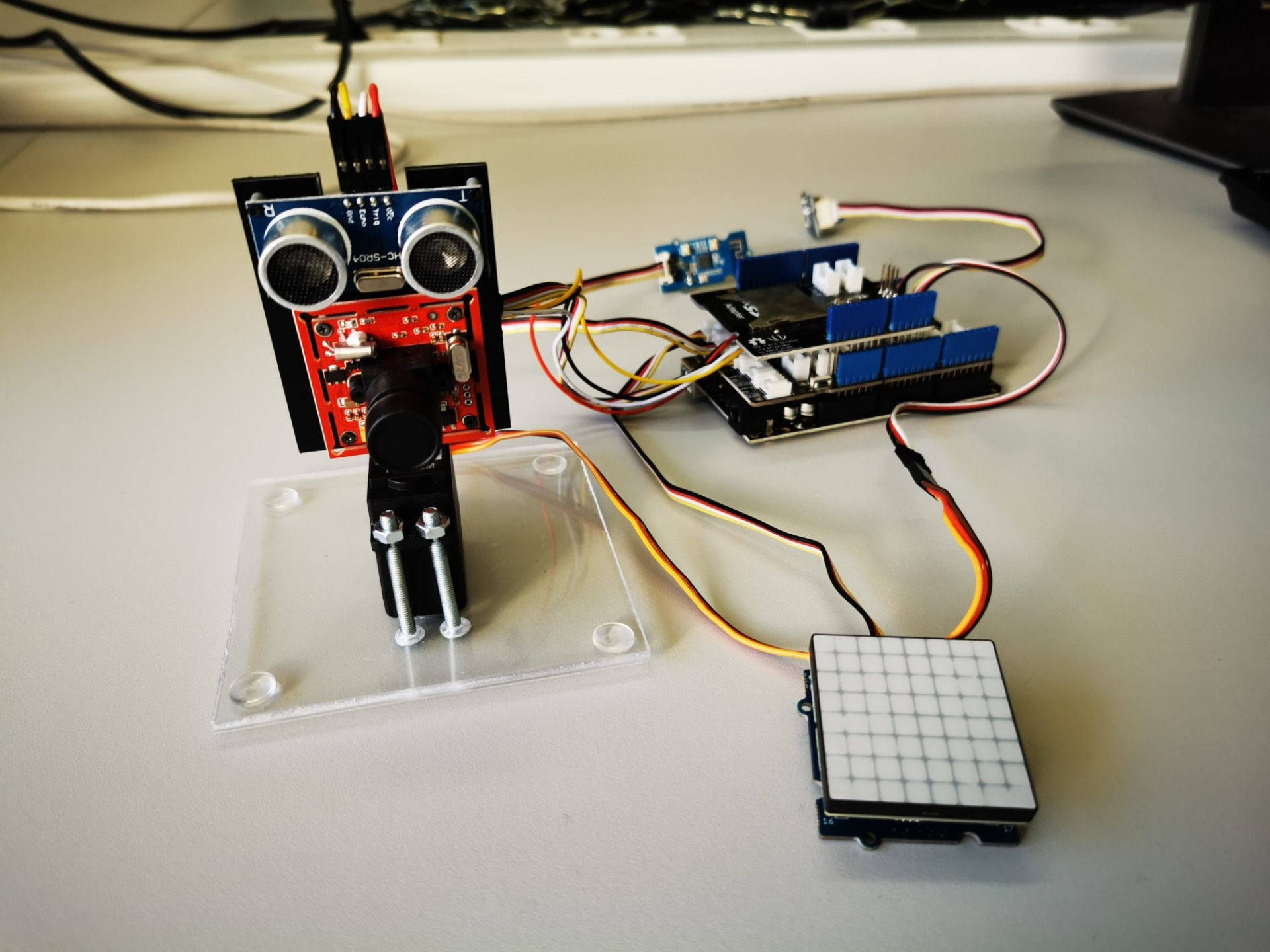
- Au total, ce projet nous couterait (en 2022) :

Carte Arduino ATmega2560	23,50€
Shield Mega v1.2	11,40€
Camera Série Grove	33,80€
Shield RGD LED matrix driver	17,95€
Capteur à ultra-son HC-SR05	6,90€
Module Grove UART Wifi v2	15,95€
Module BP ST043	1,90€
Module LED rouge grove	2,10€
Servo-Moteur FS5109	13,90€

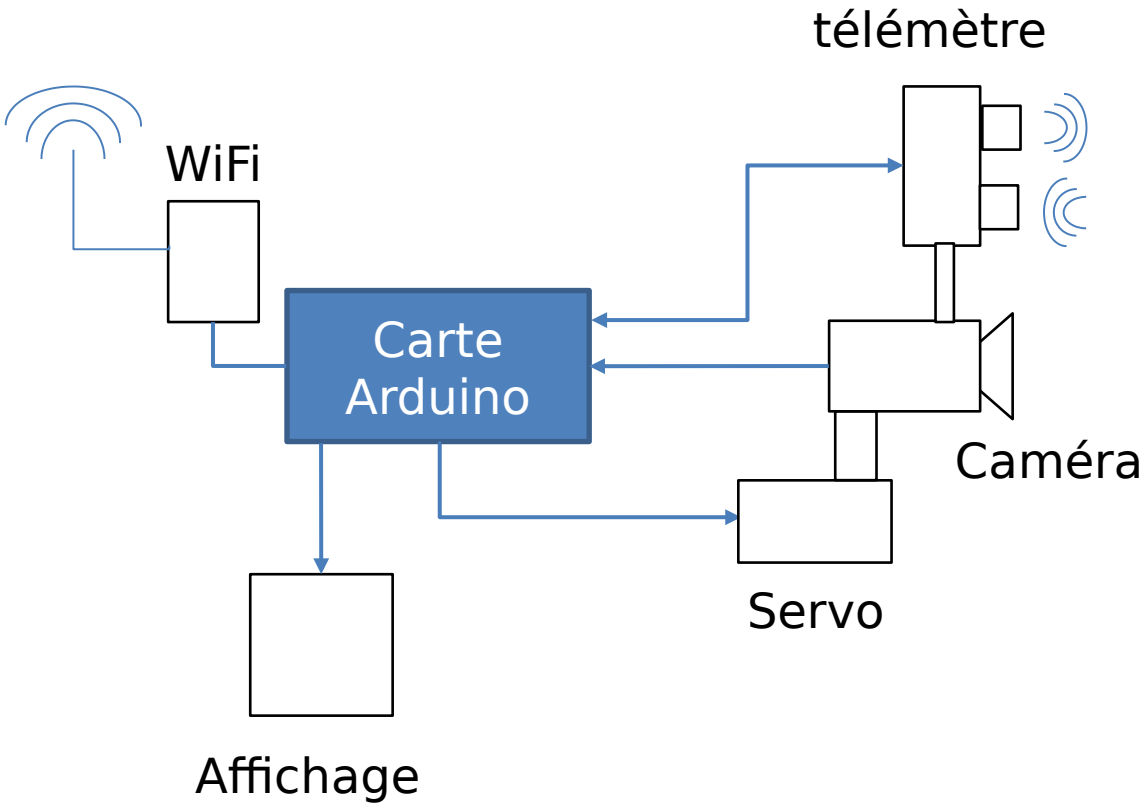
- Au total : 127,40€

Le projet que vous devrez faire

- L’idée de ce projet est de partir de votre connaissance de la programmation en assembleur Arduino, de votre connaissance de la programmation en C et de ces cours qui vous permettront de savoir quelles sont les spécifications et les exemples d’utilisation de différentes cartes ou shields pour construire en autonomie un projet assez complexe
- Le projet que l’on vous propose de faire est une caméra de surveillance qui détecte et qui avertit, lorsqu’une personne se trouve devant la caméra, un centre de surveillance par appel WiFi
- Cette caméra sera fixée sur un servo qui lui permettra de surveiller toute une zone (approximativement de 0° à 180°)



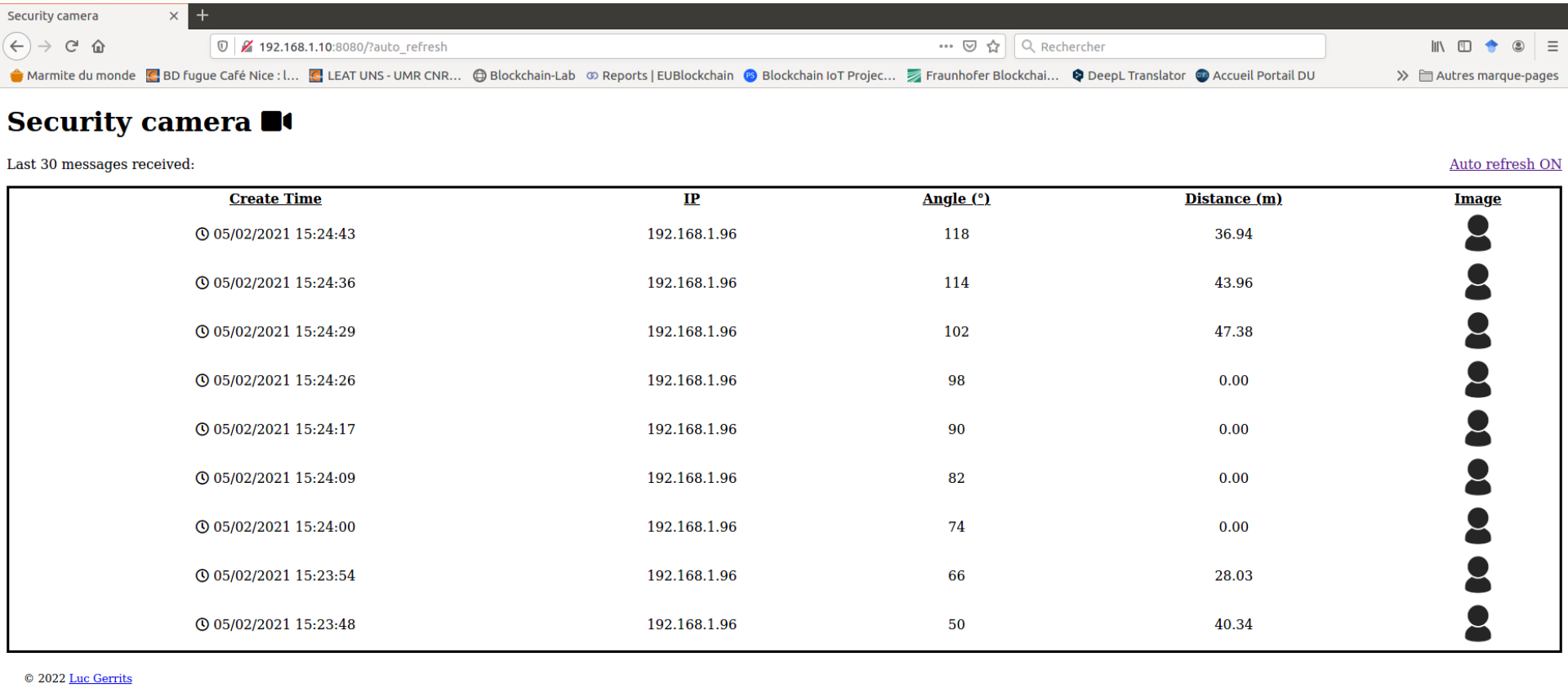
Caméra de surveillance



- Vous allez pouvoir tranquillement faire un projet qui devrait être une **caméra** qui disposerait d'un **dispositif d'alerte sur réseau WiFi** lorsque une personne apparaîtra sur l'image

- Cette caméra sera destinée à surveiller une pièce particulière et **mesurera les distances vis-a-vis des objets visibles**

Le site WEB associé à la caméra :



Le projet que vous devrez faire

- L’ensemble servomoteur + caméra + télémètre à ultrason seront fixés sur l’axe du servomoteur de telle sorte que l’axe de la caméra soit identique à l’axe du télémètre
- Vous devrez faire l’ensemble du programme (en sketch, C, C++) qui permettra de faire l’initialisation de toutes les cartes ou *shields* que vous devrez utiliser, d’une part, et les fonctions qui devront être écrites pour gérer tout cet ensemble
- A l’issue de ces séances, vous devrez présenter oralement votre travail, quel qu’en soit le résultat, à l’occasion d’une séance de présentation orale

Le projet que vous devrez faire

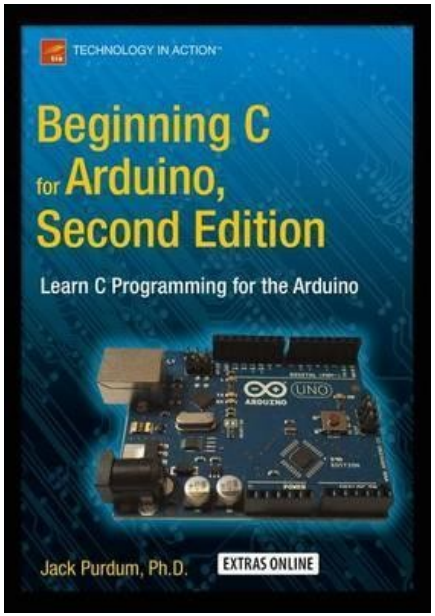
- Pour construire et pour programmer cet ensemble, vous devriez avoir tout le temps nécessaire dans les séances de projet.
- Ceci dit, et pour les étudiants qui auraient un peu de mal à comprendre les bases de la programmation des systèmes embarqués ou bien pour ceux qui auraient du mal à terminer leur projet dans les temps, il va y avoir au Fablab de Valrose (bâtiment d'enseignement de chimie) quelques plateformes complètes afin que vous puissiez continuer à développer vos programmes en autonomie.

2. L’outil de développement (IDE : Integrated Design Environment) développé par Arduino

2. L’outil de développement IDE ARDUINO

- Nous allons faire un très rapide tour des fonctionnalités de l’IDE (Integrated Development Environment) ARDUINO afin de commencer ces cours et ces projets
- En effet, après avoir programmé les micro-contrôleurs Arduino (et notamment le processeur AVR ATmega2560) en **assembleur**, nous allons passer à une autre étape qui sera la programmation en langage **sketch**, qui est quasiment du langage **C++**

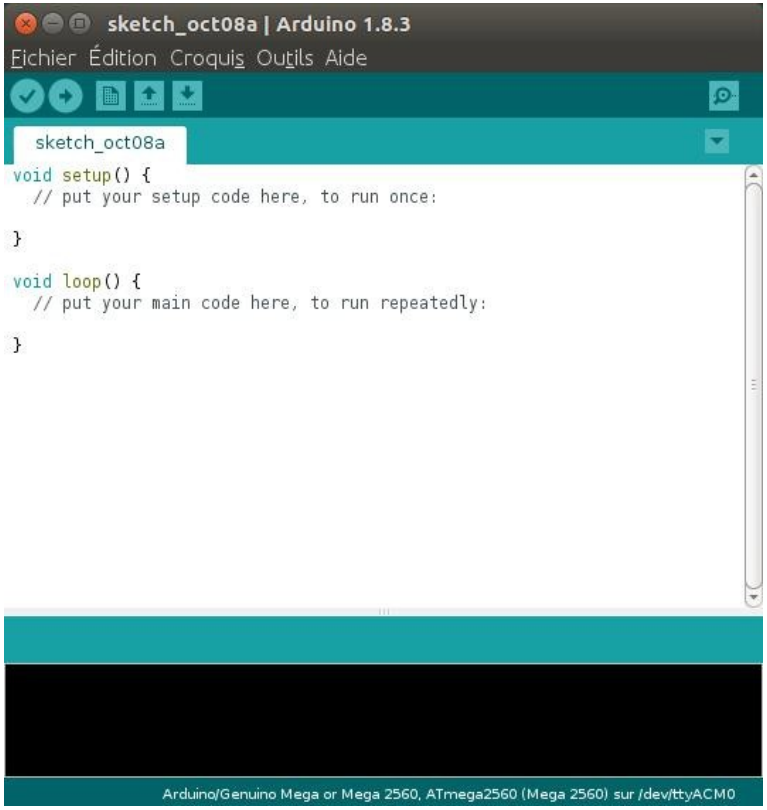
- Je vous conseille très vivement de lire l’excellent ouvrage « Beginning C for ARDUINO, Second Edition » de Jack Purdum (Ph.D.) chez Apress (ISBN : 978-1-4842-0941-7)



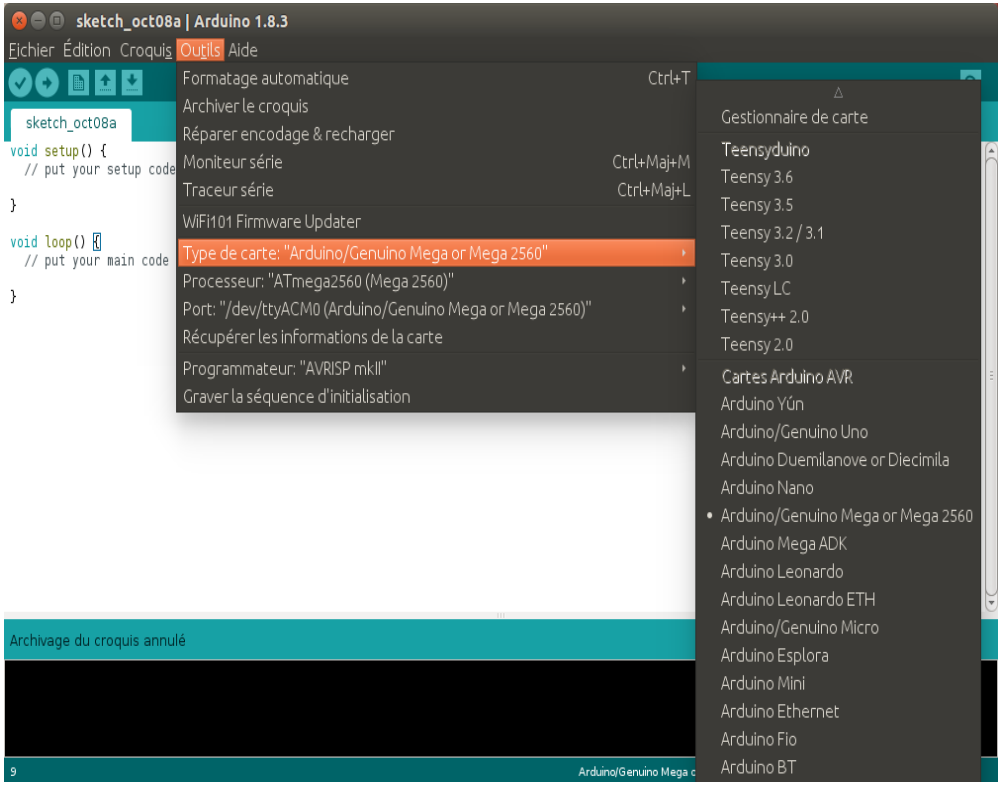
L’IDE Arduino

- Un microprocesseur sans logiciel est quasiment aussi utile qu’un vélo (non-électrique) sans pédales
- Comme chaque ordinateur, un microprocesseur a besoin d’une liste d’instructions pour faire quelque-chose. Arduino propose donc un ensemble de logiciels (qui consiste en une programmation en une version très peu modifiée de C++) avec leur outil gratuit IDE
- Ce que vous avez à faire est d’installer sur votre ordinateur cet outil IDE à partir de http://arduino.cc/en/main/software
- Je vous conseille, afin de pouvoir utiliser sans aucun problème les différentes bibliothèques que vous devrez installer également, d’installer la version 1.8.13. Elle fonctionne sur Windows, MAC OS X ou Linux (Pentium ou ARM)

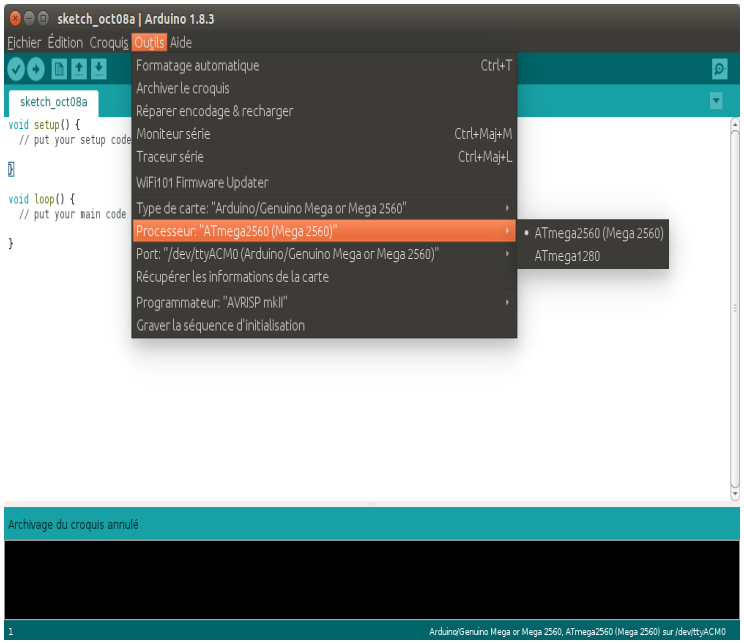
- Après avoir lancé cet outil, vous devriez voir à l'écran une image très ressemblante à celle-ci
- Vous pouvez noter que l'environnement vous affiche automatiquement les deux fonctions vides *setup()* et *loop()*.
- Ces deux fonctions *sketch* sont absolument nécessaires au fonctionnement des micro-contrôleurs Arduino quels qu'ils soient. Nous verrons à quel moment ces fonctions seront exécutées un petit peu plus tard
- Vous pouvez donc désormais « remplir » à loisir ces deux fonctions pour écrire votre programme



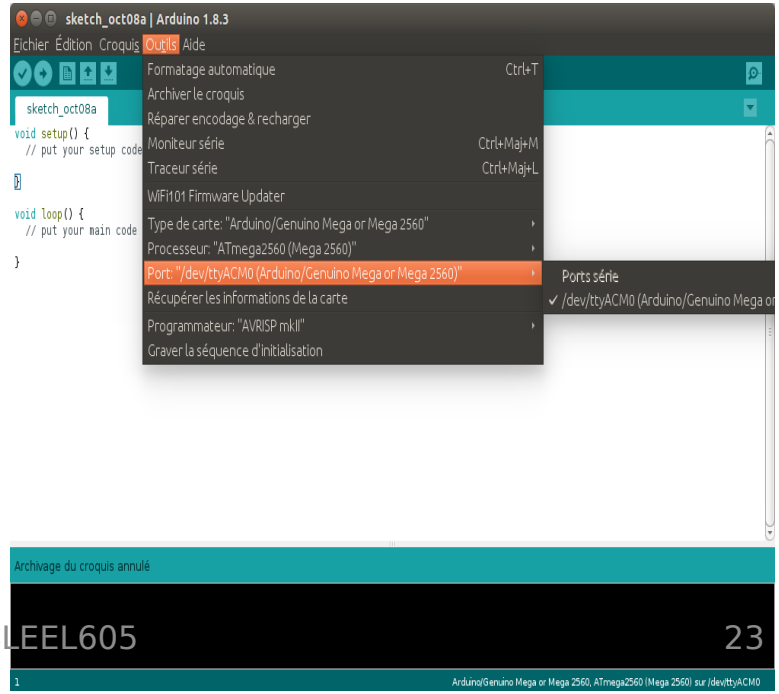
- Maintenant que vous avez l'IDE Arduino, il ne vous reste plus qu'à brancher votre carte Arduino à votre ordinateur via le cordon USB.
- Il vous faut sélectionner une carte particulière dans votre IDE, vous pouvez aller dans le menu (Tools->Type de Carte) et choisir la carte que vous venez de brancher
- Dans cet exemple, j'ai choisi une carte de type Arduino mega2560.
- C'est celle-là que vous devrez paramétrer pour faire ce projet !



- De même que pour la carte, vous devez préciser le type de micro-contrôleur que vous avez l'intention d'utiliser (ici un ATmega2560). Vous pourrez le faire en sélectionant ce processeur dans le menu (Tools->Processeur)



- Et également la connexion série que vous avez l'intention d'utiliser pour le Debug et les affichages divers et variés. Dans le cas de cet exemple et via le menu (Tools->Port) c'est une connexion de type /dev/ttyACM0 (sous Linux). Dans votre cas, vous pourrez avoir autre chose



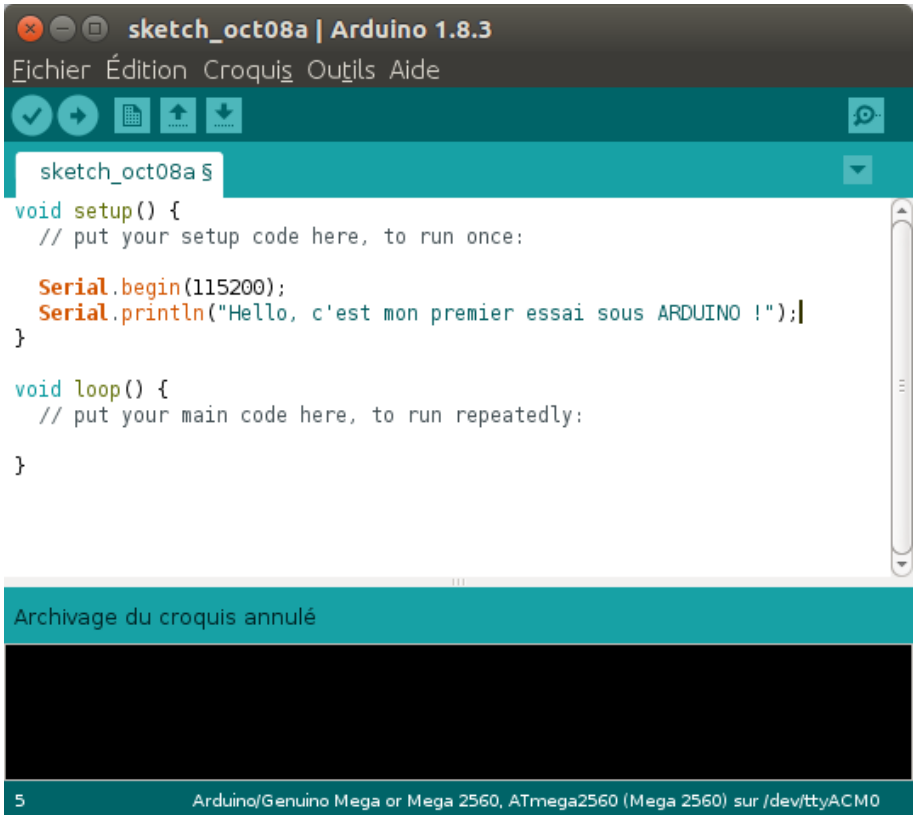
2.1) Charger et exécuter votre premier programme

- Vous pouvez maintenant écrire votre premier programme. Il vous faut tout de même savoir que votre code source va être sauvegardé dans un fichier unique, appelé un programme « *sketch* » et qui portera l'extension « .ino »
- Vous pouvez par exemple écrire les deux lignes suivante dans votre programme, comme code source de la fonction *setup()* :

```
Serial.begin(115200);
Serial.println(« Hello, c'est mon premier essai sous Arduino! »);
```

2.1) Charger et exécuter votre premier programme

- La première ligne va initialiser la liaison série à 115200 bauds (c'est-à-dire, à peu de choses près, des bits par seconde)
- La deuxième ligne va faire un affichage d'une ligne complète avec votre message

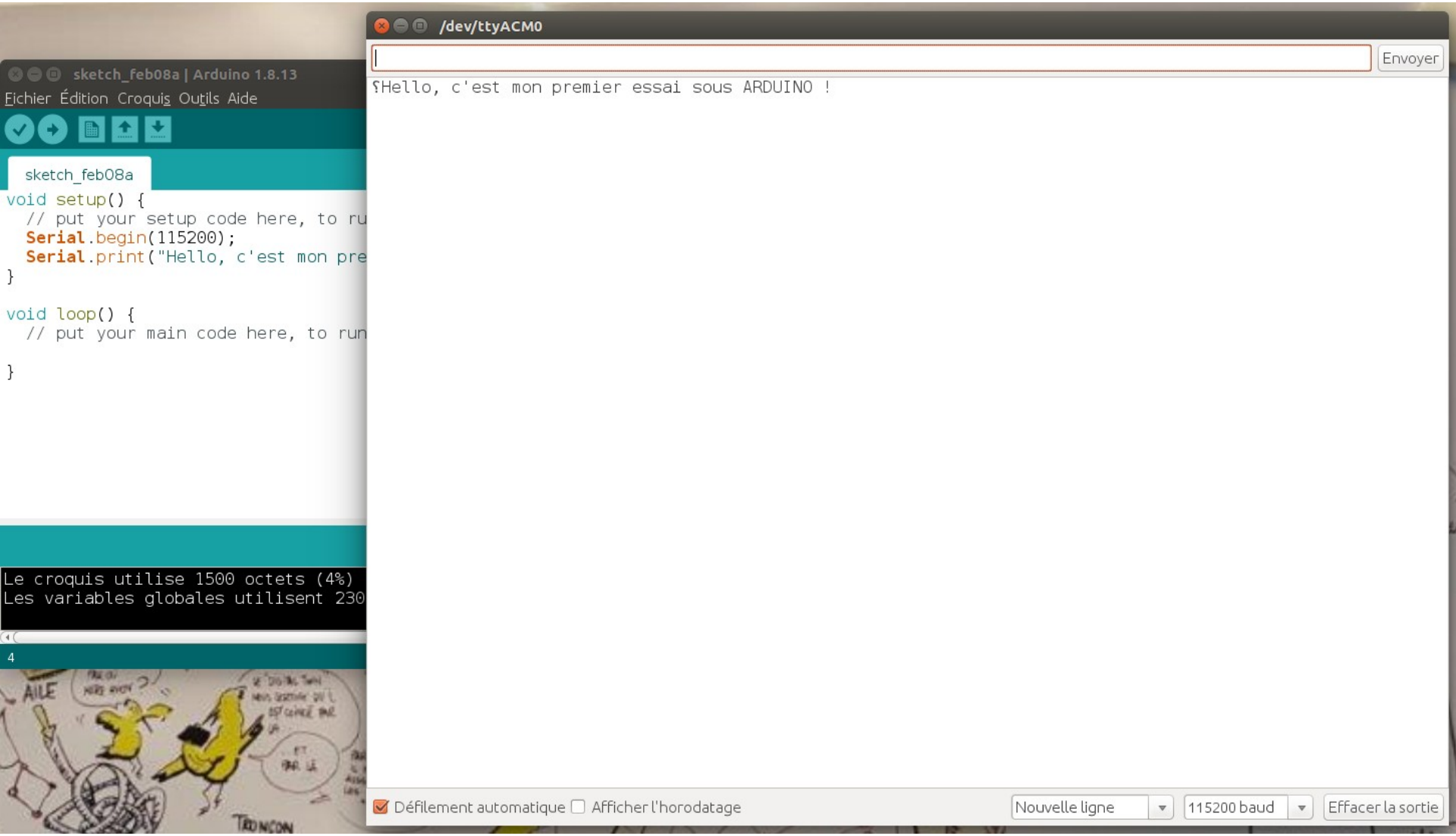


2.1) Charger et exécuter votre premier programme

- La dernière chose à faire est de compiler ce programme (le premier bouton), de vérifier qu'il n'y a pas d'erreurs de syntaxe, et de télécharger ce programme (deuxième bouton)
- Vous pourrez également ouvrir la fenêtre de l'interface série (troisième bouton)



Et vous avez la trace d’exécution de votre premier programm



2.2) Charger et exécuter votre deuxième programme

- Pour expérimenter une LED connectée à l’Arduino, il va vous falloir expérimenter les ports parallèles. Ces ports sont tous bidirectionnels et leur sens de fonctionnement est défini par le contenu d'un registre approprié. Le fait qu'une ligne soit une sortie ou une entrée du micro-contrôleur peut donc être modifié à tout instant par programme en changeant l'état d'un bit dans un registre.

- Le registre qui contrôle le sens de transfert des données s’appelle le registre DDR (pour *Data Direction Register*). Si le bit DDRx est à 0 alors le bit du port est considéré comme une entrée, si le bit DDRx est à 1, alors ce sera une sortie

- Un port X est donc un port parallèle à N bits (N dépendant du port et du circuit mais étant toujours inférieur ou égal à 8 bits). Il fonctionne grâce à trois registres accessibles dans l'espace mémoire des entrées/sorties :
  - Le registre de données PORTX,
  - Le registre du sens de transfert DDRX,
  - Et le registre d'entrée du port PINX uniquement accessible en lecture.

2.2) Charger et exécuter votre deuxième programme

- Vous pourrez également expérimenter l'allumage et l'extinction d'une LED qui serait branchée sur votre carte Arduino (par exemple sur la broche D2)
- Afin de préparer les ports parallèles pour cela, vous allez programmer correctement a la fois le registre DDRx et le registre PORTx
- La programmation du DDR se fait, en général, dans la fonction setup(), grâce à l'instruction :
  - pinMode(n° de broche, OUTPUT);
- La programmation du registre PORT se fait de la manière suivante :
  - digitalWrite(n° de broche, HIGH); // pour allumer la led
  - digitalWrite(n° de broche, LOW); // pour l'éteindre

2.3) Charger et exécuter votre troisième programme

- De même, vous pouvez faire des traitements différents selon la position d'un micro-interrupteur qui serait branché par exemple sur l'entrée D6, et ce d'une façon extrêmement facile :
  - Dans la fonction setup(), il vous faut mettre l'instruction qui initialise cette broche comme une entrée:
    ```
    pinMode(6, INPUT);
    ```
  - Dans la fonction loop() par exemple, vous pouvez y mettre:
    ```
    if (digitalRead(6) == HIGH) {
        // ici votre fonction
    }
    ```

```
int ledPin = 2;  // LED connected to digital pin 2
int inPin = 6;    // pushbutton connected to digital pin 6
int val = 0;      // variable to store the read value

void setup() {
  pinMode(ledPin, OUTPUT);  // sets the digital pin 2 as output
  pinMode(inPin, INPUT);    // sets the digital pin 6 as input
}

void loop() {
  val = digitalRead(inPin);   // read the input pin (program the PIN)
  digitalWrite(ledPin, val);  // sets the LED to the button's value
                              // (program the PORT)
}
```

Afin de connaître ces fonctions…

- Afin d'apprendre les instruction minimales pour commencer à programmer les cartes Arduino, vous pourrez aller sur le site officiel à l'adresse : https://www.arduino.cc/reference/fr/

- Vous y trouverez la quasi-totalité des fonctions d'écriture et de lecture des entrées-sorties, les fonctions qui permettent de faire des temporisations programmables, les fonctions mathématiques simples, les fonctions sur des caractères, etc.

- Vous pourrez y trouver également une aide sur les différents types de variables, sur les structures particulières de code ou sur les opérateurs de comparaison.

Pour commencer

- Afin de travailler sur le projet d'informatique embarquée sur les systèmes connectés, il vous faut télécharger au minimum un certain nombre de librairies
- Ces librairies sont des fichiers C qui vont être très utiles pour programmer et utiliser certaines cartes spécifiques
- Afin de gérer les librairies, vous devez aller dans le menu de l'outil IDE Arduino et sélectionner Outils -> Gérer les biliothèques
- Il faudra télécharger les librairies
  - Servo 1.1.6 (par Michael Margolis, Arduino) pour permettre de gérer les servo-moteurs,
  - TimerInterrupt 1.0.2 (par Khoi Hoang) pour permettre de lancer des fonctions automatiquement lorsque il y aura des interruptions de timers,
  - Arduino WiFi library for ESP8266 (WiFiEsp 2.2.2) par bportaluri qui va permettre de gérer correctement la carte WiFi

3. Les principes de fonctionnement des périphériques spécialisés

3. Les périphériques particuliers

1. Les liaisons série
2. Les timers
3. Les sorties PWM
4. La commande de sorties pour servo-moteurs

3.1 Les liaisons série

- Sur les cartes Arduino mega, mais surtout dans les micro-contrôleurs ATmega2560, vous pouvez communiquer avec d'autres ordinateurs au travers de 4 liaisons série RS232. Ce nombre dépend évidemment très fortement du micro-contrôleur que vous utiliserez
- Sur les cartes Arduino Uno (AVR328P) par exemple, il n'y en a qu'une seule

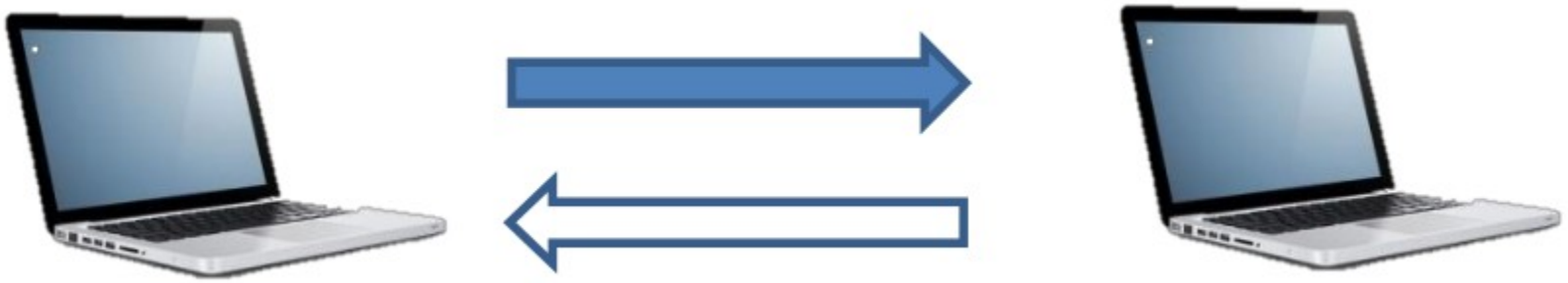
Nature des liaisons

- SIMPLEX (UNIDIRECTIONNELLE). Dans cette liaison, l’information circule toujours dans le même sens, de l’émetteur vers le récepteur (exemple : radio).



Nature des liaisons

- HALF-DUPLEX (BIDIRECTIONNELLE ALTERNÉE). C’est une liaison bidirectionnelle alternée. Les données passent dans un sens puis dans l'autre (exemple : talkie-walkie)

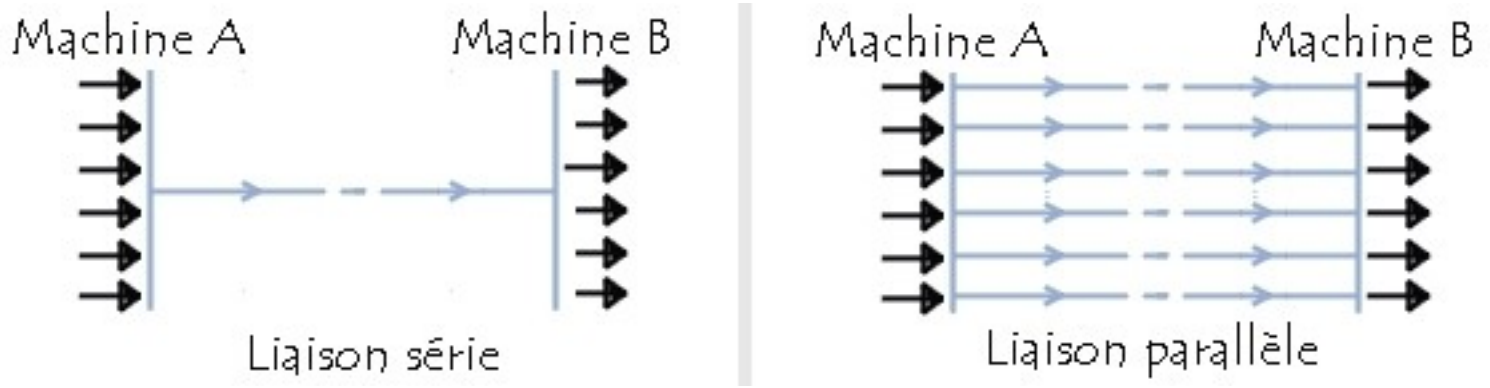


Nature des liaisons

- FULL-DUPLEX (BIDIRECTIONNELLE SIMULTANÉE). Les données s'échangent dans les deux sens (exemple : téléphone)



- Dans une liaison en série, les données sont envoyées bit par bit sur la voie de transmission. Toutefois, étant donné que la plupart des processeurs traitent les informations de façon parallèle (sous forme d'octets par exemple), il s'agit de transformer des données parallèles en données série au niveau de l'émetteur, et inversement au niveau du récepteur.
- Ces opérations sont réalisées grâce à un contrôleur de communication (la plupart du temps une puce UART, *Universal Asynchronous Receiver Transmitter*).



- Pour lire les données, le récepteur doit connaître la fréquence de l'horloge émettrice, ce qui peut être réalisé de deux façons différentes :
  - **Transmission synchrone** : le signal d'horloge est transmis par l'émetteur. Il peut être transporté par un fil, reconstitué à partir du spectre du signal de données, ou encore reconstitué à partir de caractères de synchronisation insérés au début des trames.
  - **Transmission asynchrone** : le récepteur possède une horloge interne qu'il doit synchroniser sur la séquence de bits reçue.

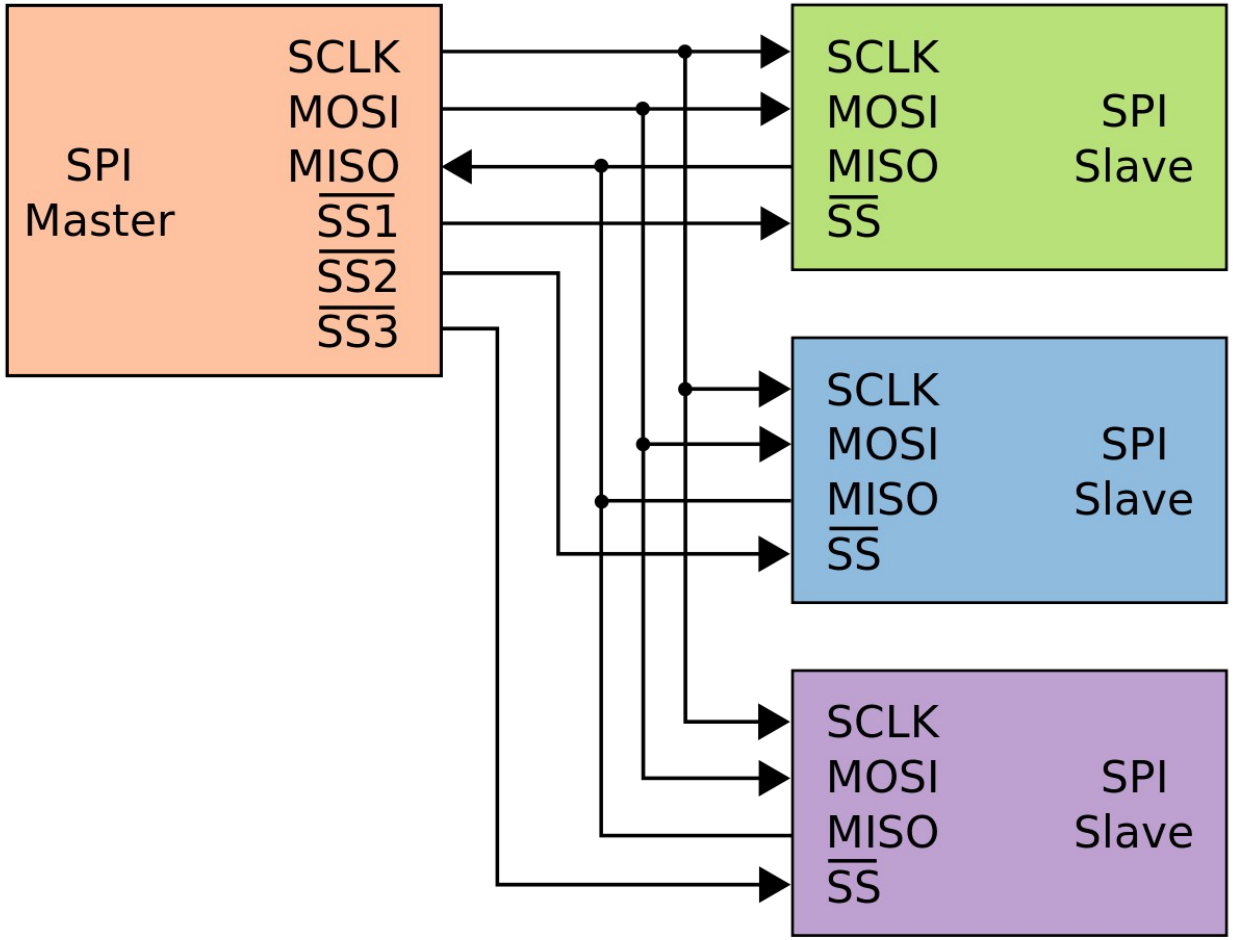
Transmission synchrone

- Le mode synchrone dédie un fil pour la synchronisation, c'est le signal d'horloge (clock)
- Dans cette liaison, l’émetteur et le récepteur sont cadencés à la même horloge. Le récepteur reçoit de façon continue (même lorsque aucun bit n'est transmis) les informations au rythme où l'émetteur les envoie
- C'est pourquoi il est nécessaire qu'émetteur et récepteur soient cadencés à la même vitesse. De plus, des informations supplémentaires sont insérées afin de garantir l'absence d'erreurs lors de la transmission

Transmission synchrone

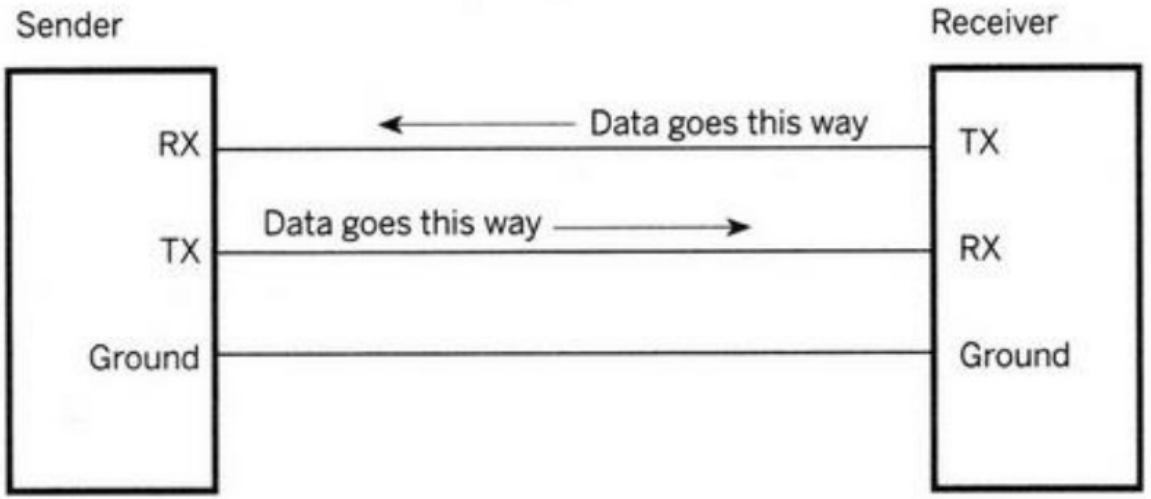
- Une liaison SPI (pour *Serial Peripheral Interface*) est un bus de données série **synchrone** baptisé ainsi par Motorola, au milieu des années 1980 et qui opère en mode full-duplex
- Les circuits communiquent selon un schéma maître-esclave, où le maître contrôle la communication. Plusieurs esclaves peuvent coexister sur un même bus, dans ce cas, la sélection du destinataire se fait par une ligne dédiée entre le maître et l'esclave appelée « Slave Select (SS) » ou « Chip Select (CS) »

Transmission synchrone



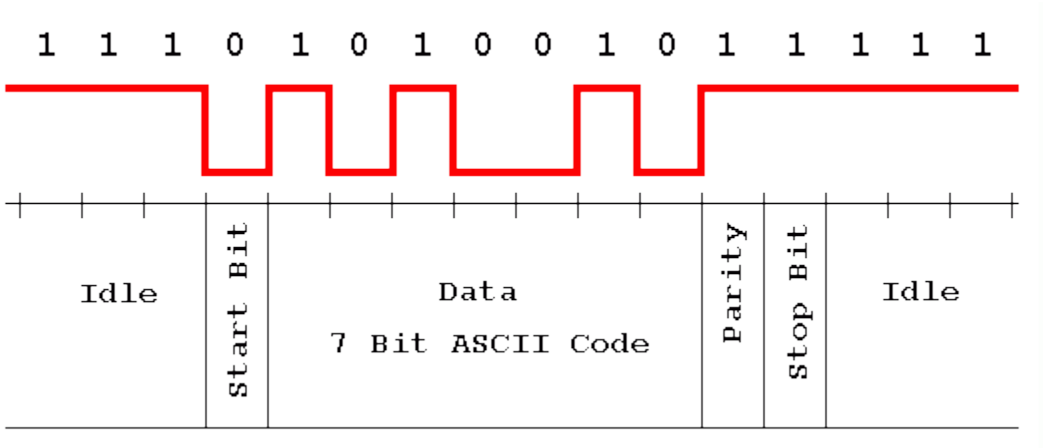
Transmission asynchrone

- Dans le mode asynchrone, il s'agit de se mettre d'accord à la connexion sur une vitesse de transmission, la synchronisation se faisant ensuite par les bits de Start et de Stop
- Toute l'astuce d'une liaison série asynchrone repose sur la forme des signaux envoyés, signaux qui permettent une synchronisation du récepteur sur chaque donnée reçue

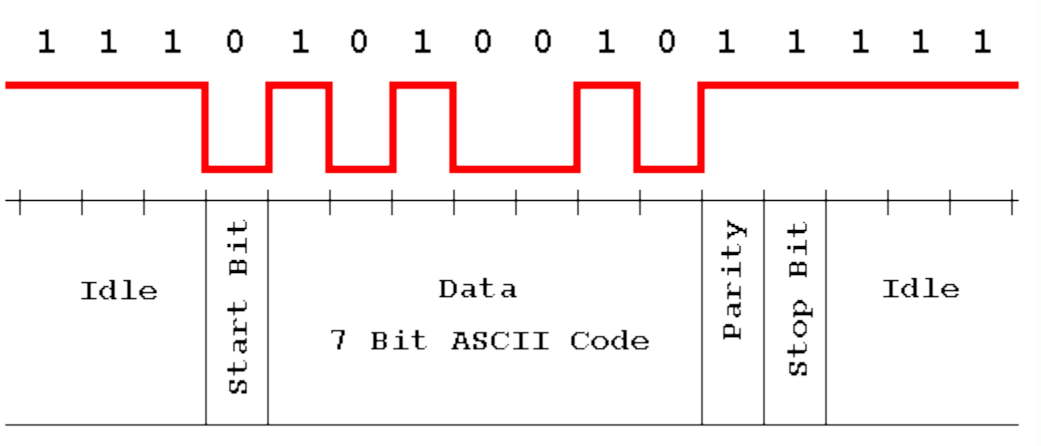


Transmission asynchrone

- Sur la figure ci-dessous on représente la transmission asynchrone de l'octet 01010010 (caractère ASCII "R", valeur décimale 82). Au repos (idle) la ligne de transmission est à l'état logique haut



Transmission asynchrone



- La transmission débute par le passage à 0 de cette ligne pendant une période de l'horloge de transmission ce qui constitue le bit de Start (ce qui signifie début ou départ)
- Les bits du mot à transmettre sont ensuite envoyés derrière ce bit de Start comme dans une transmission série synchrone
- Après le dernier bit utile, la ligne passe à nouveau à l'état haut pendant une ou deux périodes d'horloge pour constituer ce que l'on appelle le ou les bits de Stop

Vitesse de transmission

- Vitesse de transmission (Baud)
- La vitesse de transmission est mesurée en bauds, qu'il ne faut pas confondre avec le débit binaire qui mesure la quantité d'informations en bits par seconde (bps). Le baud est le nombre de symboles transmis physiquement par seconde, alors que le bit désigne une unité d'information
- La confusion vient du fait qu'au début les modems transmettaient seulement 1 bit par baud, le débit en bits par seconde était donc équivalent au nombre de bauds
- Le terme "baud" provient du patronyme d’Émile Baudot, l’inventeur du code Baudot utilisé en télégraphie.

Vitesse de transmission

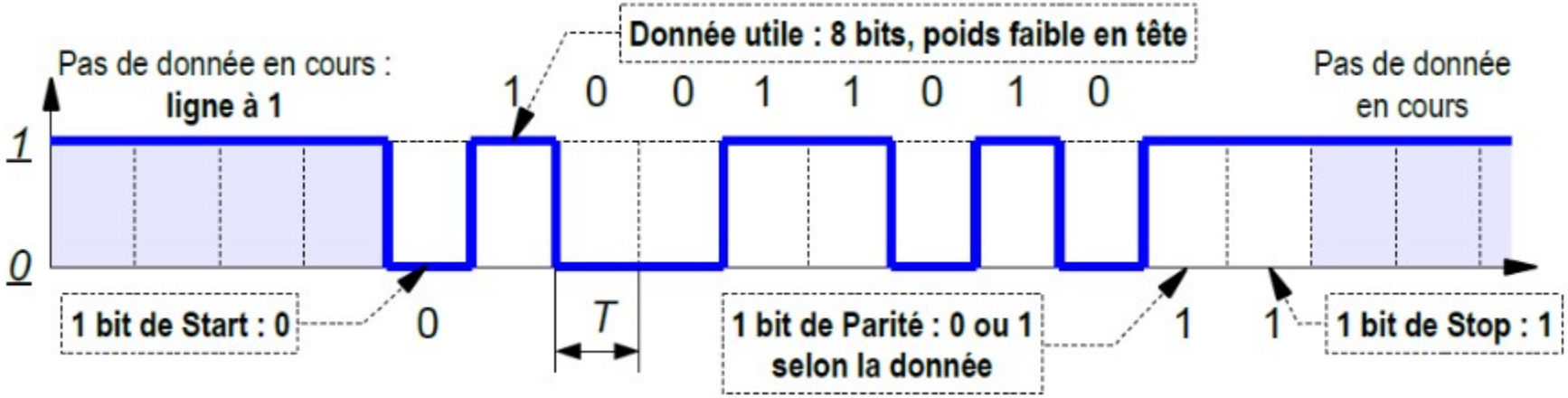
Vitesse en bits par seconde (bps)	*Application*
75	Émission Clavier Minitel → Serveur Télétexte
110, 300, 600	
1200	Réception Serveur Télétexte → Écran Minitel
2400	
4800, 9600, 14400, 19200	API, Modem-Fax (14400)
56000, 115200, 128000, 256000	
187,5 kbps , 1,5 Mbps, 10 Mbps	Bus de terrain : Profibus, ...

- Dans notre cas, nous utiliserons des vitesse relativement élevées (115200 bps)

Étude de la trame

- Le protocole d'échange asynchrone est défini par l’envoi, pour chaque caractère émis, de :
  - un bit de Start,
  - les 5 à 8 bits de données, poids faible en tête,
  - éventuellement, un bit de vérification de Parité qui permet de déceler des erreurs de transmission des 8 bits de donnée sur la ligne
  - 1, 1 ½, ou 2 bits de Stop après.

Etude de la trame



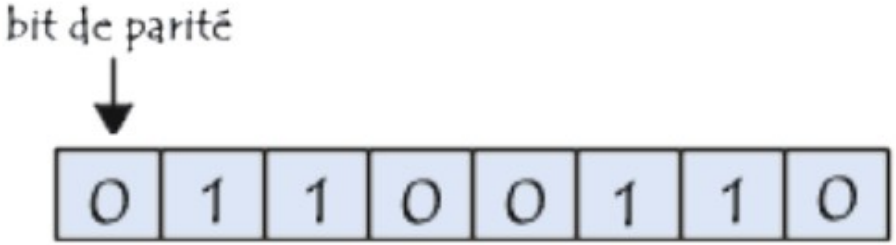
- Lorsqu'aucun caractère ne circule sur la ligne, celle-ci reste à l'état logique haut (« 1 »).
  - « 1 » logique = 5V devient une tension comprise entre -3V et -25V sur les protocoles RS232
  - « 0 » logique = 0V devient une tension comprise entre +3V et +25V sur les protocoles RS232

Contrôle de parité

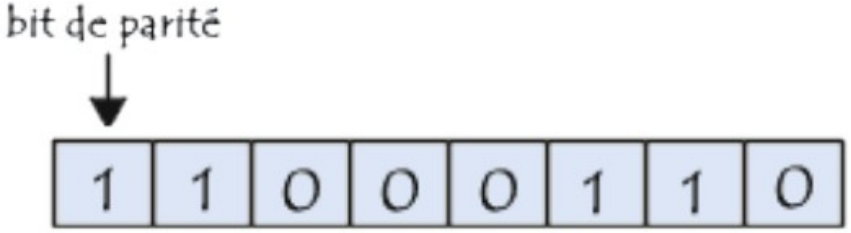
- Le contrôle de parité (appelé parfois VRC, pour Vertical Redundancy Checking) est un des systèmes de contrôle des erreurs de transmission les plus simples
- Il consiste à ajouter un bit supplémentaire (appelé bit de parité) à un certain nombre de bits de données appelé mot de code (généralement 7 bits, pour former un octet avec le bit de parité) dont la valeur (0 ou 1) est telle que le nombre total de bits à 1 soit pair (ou impair)
- Pour être plus explicite, dans un mode de contrôle à parité paire, cela consiste à ajouter un 1 si le nombre de bits du mot de code est impair, 0 dans le cas contraire

Contrôle de parité

- Dans le cas d’une parité paire, prenons l'exemple suivant :

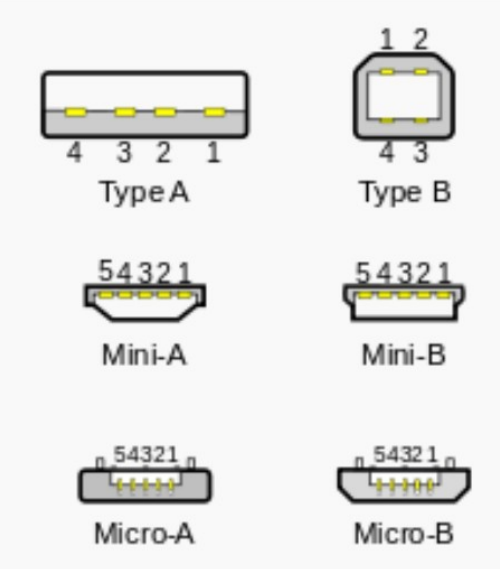


- Dans cet exemple, le nombre de bits de données à 1 est pair, donc le bit de parité est positionné à 0. Dans l'exemple suivant par contre, les bits de données étant en nombre impair, le bit de parité est à 1 :



Liaison série sur bus USB

- Le bus USB utilise 4 fils, ainsi que des connecteurs particuliers.
- Deux des fils permettent d'acheminer l'alimentation électrique du périphérique (une souris USB par exemple), et deux fils véhiculent les données elles-mêmes



Liaison série sur bus USB

- Brochage (Types A et B)

Fonction	Couleur	Numéro de broche pour les types A et B	Numéro de broche pour le type mini B
Alimentation +5 V (VBUS)	Rouge	1	1
Données (D-)	Blanc	2	2
Données (D+)	Vert	3	3
Masse (GND)	Noir	4	5[30]

- Brochage (Type C)



A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1
GND	RX2+	RX2-	VBus	SBU1	D-	D+	CC	VBus	TX1-	TX1+	GND
GND	TX2+	TX2-	VBus	VConn			SBU2	VBus	RX1-	RX1+	GND
B1	B2	B3	B4	B5	B6	B7	B8	B9	B10	B11	B12

Débit maximum possible sur port USB

- Lorsque l'on parle d'un équipement USB, il est nécessaire de préciser la version de la norme (1.1, 2.0 ou 3.0) mais également la vitesse (low/full/high/super speed).

Version	USB 1.0	USB 1.1	USB 2.0	Wireless USB	USB 3.0	USB 3.1
Année	1996	1998	2000	2005	2008	2013
Débit	1,5 Mbit/s 0,19 Mo/s	12 Mbit/s 1,5 Mo/s	480 Mbit/s 60 Mo/s	480 Mbit/s 60 Mo/s	5 Gbit/s 600 Mo/s	10 Gbit/s 1,2 Go/s

Liaisons série sur Arduino

- Dans un processeur Arduino, et sous l'environnement IDE, il est possible d'utiliser la liaison série (de l'UART 0) grâce au type Serial. Celle-ci doit s'initialiser avec, au minimum, la vitesse de transmission :

```
Serial.begin(115200); // ici la vitesse est choisie à 115200 bauds
```

- On peut dès lors envoyer des messages sur cette liaison série :

```
Serial.print(« A »);
Serial.print(value);
Serial.print("Bonjour !");
```

Liaisons série sur Arduino

- On peut également afficher des lignes complètes, les caractère retour chariot (\r) et linefeed (\n) seront affichés :

  Serial.println(« Ceci est un message ! »);

- On peut également lire des données sur la liaison entrante :

  int a = Serial.read(); // pour lire un octet sur la liaison serie

- Pour compter les caractères reçus dans le tampon d’entrée :

  int cnt = Serial.available();

- Pour vider les caractères reçus :

  Serial.flush();

Liaisons série sur Arduino

- Pour le processeur ATmega2560, il y a 4 liaisons séries, les liaisons 0, 1, 2 et 3. Pour les utiliser, l'environnement IDE propose tout simplement les classes :

```
Serial        // Liaison n°0
Serial1       // Liaison n°1
Serial2       // Liaison n°2
Serial3       // Liaison n°3
```

Liaisons série sur Arduino

- On peut utiliser également la librairie SoftwareSerial, qui est une librairie par défaut de Arduino, qui vous permet d'employer des fonctions un peu différentes
- Vous pouvez, grâce à cette librairie, créer des objets liaison série :

```
#include <SoftwareSerial.h>
// Pour préciser les connexion RxD et TxD
SoftwareSerial   MaConnnexionSerie(15, 14);
```

3.2 Les timers

- Les timers fonctionnent en incrémentant un compteur. Ce registre peut compter jusqu'à un certain nombre, suivant sa taille. Le timer incrémente ce compteur jusqu'à ce qu'il atteigne sa valeur maximale, ce qui provoque un débordement de capacité (Overflow), et se réinitialise à zéro

3.2 Les timers

- Normalement le timer active un bit particulier (flag) pour faire savoir qu'il y a eu débordement. On peut vérifier l'état de ce bit manuellement, ou on peut paramétrer une interruption pour qu'elle ait lieu dès que le flag est activé

- Comme pour n'importe quelle autre interruption, on peut spécifier une routine d'interruption (ISR) pour exécuter le code de notre choix lorsque le timer déborde. L'ISR réinitialisera le flag en même temps qu'elle s'exécutera, c'est donc la solution la plus simple et la plus rapide

Timers - Exemple

```
#define TIMER_INTERRUPT_DEBUG      0
#define USE_TIMER_1     true
#define USE_TIMER_2     true
#define USE_TIMER_3     true
#define USE_TIMER_4     true
#define USE_TIMER_5     true
#include “TimerInterrupt.h”
void TimerHandler1(void){
    // Vous pouvez placer ici un code quelconque
}
#define TIMER1_INTERVAL   1000
Void setup(){
    Serial.begin(115200);
    Serial.println(« Debut…. »);

    ITimer1.init();
    if(ITimer1.attachInterruptInterval(TIMER1_INTERVAL, Timer1Handler))
        Serial.println(« Starting Timer1… »);
    else
        Serial.println(« Problem with Timer1…! »);
}

Void loop(){
}
```

Timers (suite)

- Pour incrémenter le compteur à intervalles réguliers, le timer doit avoir accès à une horloge. Celle-ci génère un signal se répétant périodiquement. À chaque fois que le timer détecte ce signal, il incrémente une fois le compteur
- Parce que les timers sont dépendants de l'horloge, la plus petite unité de temps mesurable sera la période de cette horloge. Par exemple, si l'on fournit un signal d'horloge de fréquence 1 MHz à un timer, la résolution du timer (ou période T du timer) peut être calculée comme suit :

$$T = 1/f = 1/10^{6}\ \text{Hz} = 1.10^{-6}\ \text{s}$$

Types de Timers

- L'ATmega168 et l'ATmega328 possèdent 3 timers : Timer0, Timer1, Timer2. Ils ont aussi un timer "chien de garde" (en anglais watchdog), qui peut être utilisé comme sécurité ou mécanisme de reset logiciel. Voici quelques détails sur chaque timer :
  - Timer0 est un timer 8 bits, son compteur peut compter jusqu'à 255 (comme un unsigned byte sur 8 bits). Timer0 est utilisé nativement par l'Arduino pour des fonctions de chronométrage comme delay() et millis()
  - Timer1 est un timer 16 bits, avec un compteur allant jusqu'à 65536 (comme un unsigned int sur 16 bits). La librairie Servo de l'IDE Arduino utilise ce timer
  - Timer2 est un timer 8 bits très similaire à Timer0. Il est utilisé par la fonction tone() Arduino
  - Les ATmega1280 et ATmega2560 (comme sur les Arduino Mega) ont trois timers supplémentaires. Ce sont tous des timers 16 bits, et fonctionnent de la même manière que Timer1

Configurer et faire compter un Timer

- Pour utiliser ces timers, il faut les paramétrer, puis les démarrer. Pour ce faire, nous utilisons les registres de l'AVR qui stockent les paramètres des timers. Chaque timer possède un certain nombre de registres ayant des fonctions particulières
- Deux de ces registres stockent les valeurs de départ, et sont appelés TCCRxA et TCCRxB, où x est le numéro du timer. TCCR signifie Timer/Counter Control Register : Registre de Contrôle Timer/Compteur
- Chaque registre contient 8 bits, et chaque bit permet de stocker un paramètre

Configurer et faire compter un Timer

TCCR1A – Timer/Counter1 Control Register A

Bit	7	6	5	4	3	2	1	0	
(0x80)	COM1A1	COM1A0	COM1B1	COM1B0	–	–	WGM11	WGM10	TCCR1A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

TCCR1B – Timer/Counter1 Control Register B

Bit	7	6	5	4	3	2	1	0	
(0x81)	ICNC1	ICES1	–	WGM13	WGM12	CS12	CS11	CS10	TCCR1B
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- Les paramètres les plus importants sont les trois derniers bits dans TCCR1B : CS12, CS11 et CS10. Ce sont ceux qui donnent les réglages de l'horloge du timer. Avec différentes combinaisons, on peut demander au timer de fonctionner à différentes vitesses. Voici la table correspondante dans la documentation :

Configurer et faire compter un Timer

- Voici la table correspondante dans la documentation :

Table 15-5. Clock Select Bit Description

CS12	CS11	CS10	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	$clk_{I/O}$/1 (No prescaling)
0	1	0	$clk_{I/O}$/8 (From prescaler)
0	1	1	$clk_{I/O}$/64 (From prescaler)
1	0	0	$clk_{I/O}$/256 (From prescaler)
1	0	1	$clk_{I/O}$/1024 (From prescaler)
1	1	0	External clock source on T1 pin. Clock on falling edge.
1	1	1	External clock source on T1 pin. Clock on rising edge.

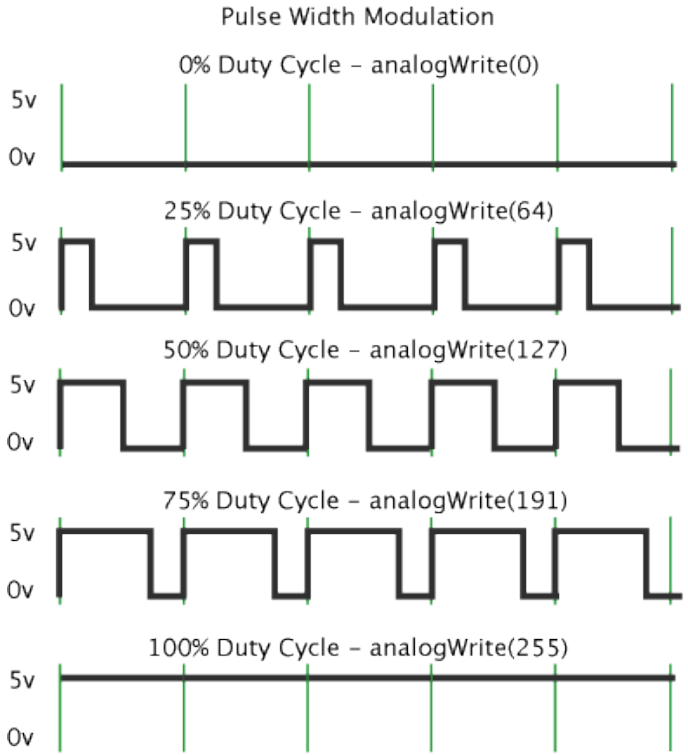
Les modes des Timers

- Les timers peuvent être configurés selon différents modes :
  - Le mode PWM (Pulse Width Modulation) – les sorties OCxy sont utilisées pour générer des signaux PWM,
  - Le mode CTC (Clear Timer on Compare match) – lorsque le compteur du timer atteint le registre de comparaison, le timer est remis à zéro

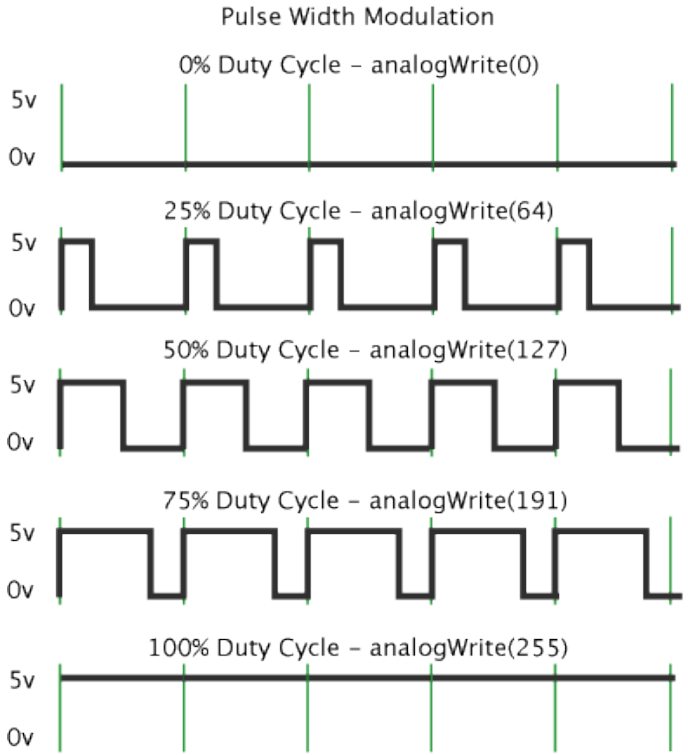
3.3 Les sortie PWM

- On va, dans cette partie, essayer d’éteindre très progressivement une LED qui serait branchée sur l’une des broches de notre carte Arduino.
- L'exemple de l‘évanouissement démontre l'utilisation de la sortie analogique (PWM) pour faire éteindre une LED.
- La modulation de largeur d'impulsion, ou *Pulse Width Modulation* (PWM), est une technique permettant d'obtenir des résultats analogiques avec des moyens numériques. La commande numérique est utilisée pour créer un signal rectangulaire, c’est-à-dire un signal commuté entre l'activation et la désactivation

- Ce schéma on-off peut simuler des tensions entre Vcc et la masse (0 Volt) en changeant la proportion du temps où le signal passe en marche par rapport au temps où le signal passe en arrêt.
- En réalité c'est la valeur moyenne de ce signal carré qui est la valeur analogique



- La durée du temps d'activation est appelée la largeur d'impulsion. Pour obtenir des valeurs analogiques variables, il faut modifier ou moduler cette largeur d'impulsion
- Si l’on répète ce schéma marche-arrêt assez rapidement avec une LED par exemple, le résultat sera comme si le signal était une tension constante entre 0 et Vcc contrôlant la luminosité de la LED



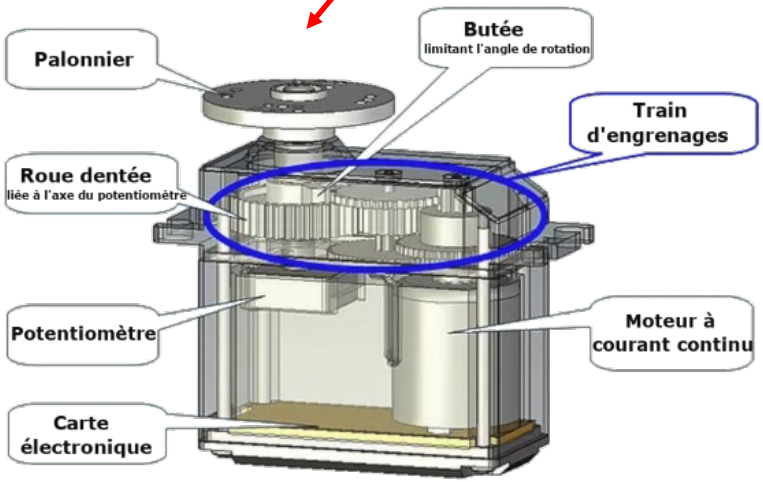
- Sur cette figure, les lignes vertes représentent une période de temps régulière. Cette durée ou période est l'inverse de la fréquence PWM. Avec une fréquence PWM de 500 Hz, la période mesurerait 2 millisecondes.
- Un appel à analogWrite() se fait sur une échelle de 0 à 255, de sorte que analogWrite(255) demande un rapport cyclique de 100% (toujours activé), et analogWrite(127) un rapport cyclique de 50% (activé la moitié du temps) par exemple.

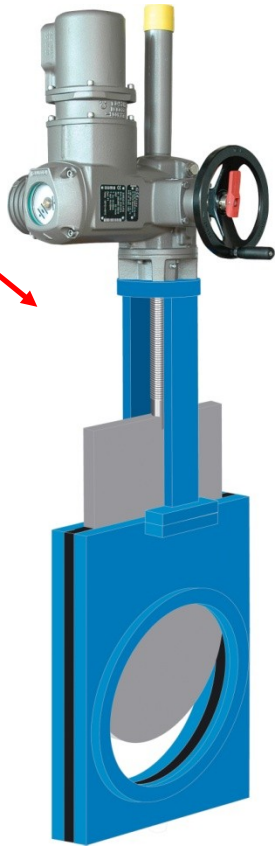
3.4 Les sorties de type « Servo »

- La première question que vous vous posez sans doute est : Qu’est-ce qu’un servo-moteur ?
- Les servo-moteurs sont des dispositifs de type moteurs électriques, qui sont capables de maintenir une certaine force, une certaine opposition, à des effort statiques et dont la position est vérifiée, en continu et corrigée en fonction de la mesure. Des dispositifs de type servo-moteurs sont donc des systèmes asservis
- Un servo-moteur intègre dans un même boitier les éléments mécaniques (moteur et engrenage généralement) et l’électronique de commande et d’asservissement du moteur

- Il existe principalement trois catégories de servo-moteur :
  - Les servo-moteurs à positionnement angulaire,
  - Les servo-moteurs à rotation continue,
  - Les servo-moteurs à déplacement linéaire.







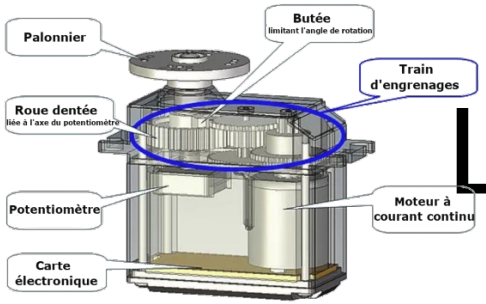
https://www.cf-techno.com

Les éléments internes d’un servo-moteur à positionnement angulaire

- un moteur électrique à courant continu (MCC)
- un train d’engrenages
- un potentiomètre relié au train d’engrenages par l’intermédiaire d’une roue dentée, pour fournir la mesure de l’angle de déviation de l’axe
- et une carte électronique pour gérer l’asservissement
- dans le servo-moteur standard, il y a une butée mécanique qui limite l’excursion en angle de -90° à +90° par rapport à la position dite neutre, soit une excursion de 180°

Le moteur à courant continu

- Le moteur à courant continu (MCC) est alimenté par une tension constante (quelques volts dans le cas des servo-moteurs de modélisme). Par principe, un MCC tourne à vitesse angulaire proportionnelle à sa tension d'alimentation, en première approximation.
- Typiquement, dans un servo-moteur standard, sous 5 V, le moteur tourne à vitesse élevée, plusieurs milliers de tours par minute.
- En revanche, le couple sur l'axe est faible. C'est la raison pour laquelle on place sur l'axe du moteur un train d'engrenages appelé aussi **réducteur**.



Le train d’engrenages

- Il s’agit d’une cascade d’étages constitués chacun d’un couple de roues dentées. Chaque étage possède son rapport de réduction qui est égal au rapport du nombre de dents des deux roues ou pignons. Au total, le rapport de réduction est le produit de tous les rapports et peut atteindre typiquement une valeur de l’ordre de 400 ou 500
- Sur un plan électromécanique, la puissance de sortie du moteur (P) est donnée par le produit :

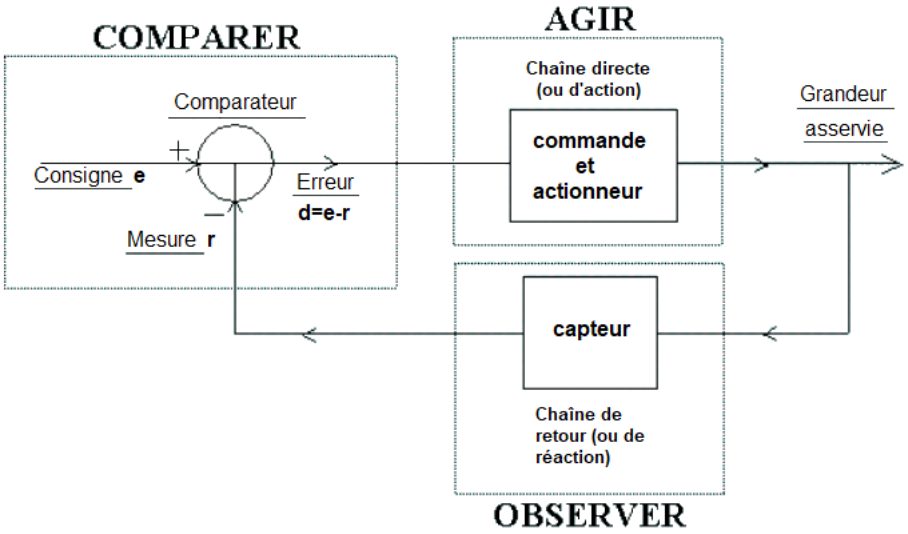
$$P=Cm.\Omega m$$

- Où Cm représente le couple sur l’axe du moteur et Ωm la vitesse angulaire de rotation de l’axe du moteur.

- Aux pertes près, le produit P=C.Ω est constant. La réduction de vitesse est ainsi associée à la démultiplication du couple dans des rapports identiques

Le potentiomètre et la carte électronique

- Le potentiomètre joue le rôle de **capteur de position** de l'axe terminal. Il délivre une tension proportionnelle à la position angulaire, tension exploitée par la carte électronique dans la boucle d'asservissement
- Pour décrire le principe très général d'une **boucle d'asservissement ou de régulation**, on peut se référer au schéma-bloc fonctionnel ci-dessous :



3.4 Les sorties de type « Servo »

- Comment peut on utiliser un servo-moteur ?



Utilisation sur Arduino

```
#include <Servo.h>


Servo MyServo;
MyServo.attach(9);                      // Un servo est « attaché » à la broche 9
MyServo.attach(9, min, max);            // Les valeurs min et max doivent etre
                                        // comprises entre 0 et 200
MyServo.write(value);                   // Value est un angle <200
MyServo.writeMicroseconds(value); // Pour une impulsion en microseconds
Int a = MyServo.read();                 // retourne un nombre qui est un
                                        // angle entre 0 and 180 degrés
int b = MyServo.readMicroseconds();  // retourne l'impulsion qui a été envoyée
                                        // au servo en microsecondes
Bool MyServo.attached();                // renvoie vrai si le servo est attaché
MyServo.detach();                       // détache le servo, lorsque c'est la fin
                                        // du programme
```